

Složeni oblik vežbe 3

Predispitna provera – domaći zadatak (5 poena)

Semafor za vozila i pešake

U VHDL jeziku za opis fizičke arhitekture potrebno je realizovati semafor za pešake i automobile koji odbrojava vreme i koji ima sledeće prolaze:

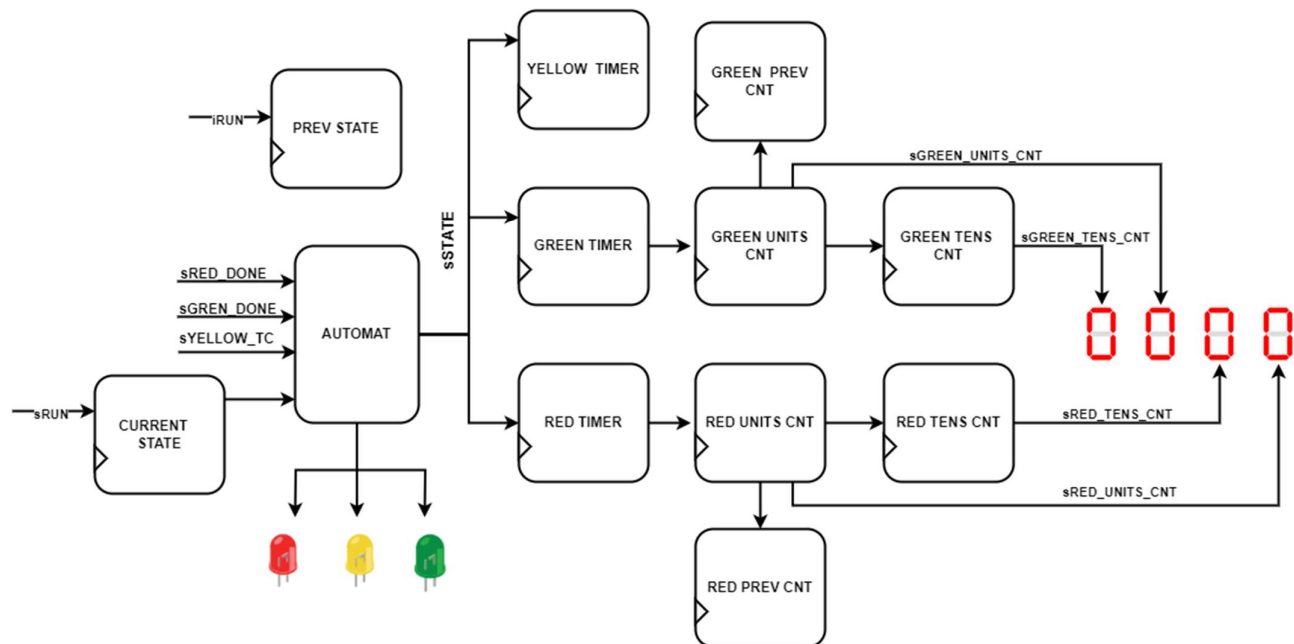
Ulazi sistema:

- ***iCLK*** – ulazni takt signal, podrazumevan u svakoj sekvencijalnoj mreži
- ***iRST*** – ulazni reset signal, aktivan na '1' i ***asinhron***, podrazumevan u svakoj sekvencijalnoj mreži
- ***iRUN*** – signal za aktivaciju rada semafora

Izlazi sistema:

- ***oRED*** – crveno svetlo semafora
- ***oYELLOW*** – žuto svetlo semafora
- ***oGREEN*** – zeleno svetlo semafora
- ***oDIS*** – izbor displeja
- ***o7SEGM*** – prikaz na 7-segmentnom displeju

Arhitektura sistema je prikazana na slici:

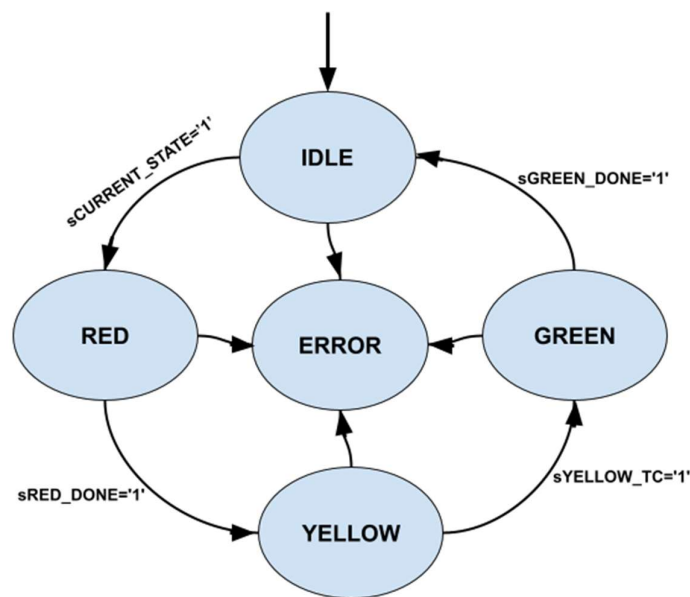


! Zarad bolje preglednosti slike, komponente za vremensko multipleksiranje i prikaz na displeju nisu prikazani na slici. Za realizaciju pogledati vežbu 5 i dodatno samostalno istražiti literaturu na ovu temu.

Sistem obuhvata sledeće komponente:

- **CURRENT STATE** – registar koji čuva informaciju da li je semafor aktiviran,
- **PREV_STATE** – registar koji čuva vrednost signala *iRUN*,
- **TIMER** – broji jednu sekundu nakon čega se resetuje,
- **UNITS_CNT** – brojač jedinica, broji dekrementalno,
- **TENS_CNT** – brojač desetica, broji dekrementalno,
- **PREV_CNT** – čuva prethodnu vrednost brojača jedinica,
- **YELLOW_TIMER** – broju jednu ipo sekundu nakon čega se resetuje,
- **AUTOMAT** – obuhvata registar stanja, funkciju prelaza i funkciju izlaza (**pogledati vežbu 6**)
- **7SEG_DEC** – dekodeer za 7-segmentni displej,
- **Komponente za vremensko multipleksiranje.**

Na slici je prikazana funkcija prelaza između stanja semafora:



! Ukoliko nije zadovoljen uslov prelaska u naredno stanje, semafor ostaje u trenutnom stanju. Ako nije aktivno nijedno stanje došlo je do greške i sistem do kraja ostaje u stanju **ERROR**.

Potrebno je simulirati semafor za vozila i pešake. Ukoliko je aktivno crveno svetlo za vozila, aktivno je zeleno svetlo za pešake i obrnuto. Kada je aktivno zeleno svetlo za pešake tajmer odbrojava **19 sekundi**, a za crveno svetlo **29 sekundi**.

Na početku, semafor se nalazi u stanju **IDLE**. Aktivira se pritiskom na dugme **iRUN**, nakon čega semafor počinje sa radom. Registar **PREV STATE** čuva informaciju o vrednosti signala **iRUN**. Kada se aktivira, semafor radi sve dok se ne resetuje.

Ukoliko je trenutna vrednost signala **sPREV_STATE** jednaka 0, a signal **iRUN** je na visokom nivou aktivira se signal **sRUN**. Registar **CURRENT STATE** čuva vrednost signala **sRUN** ukoliko je njegova vrednost 1. Takođe, sistem prelazi u stanje **RED**, nakon čega se aktivira zeleno svetlo za pešake.

TIMER odbrojava kada je semafor aktiviran i kada je stanje **RED**, a kada prođe jedna sekunda tajmer se resetuje, a brojač jedinica **UNITS_CNT** umanju svoju vrednost za jedan. Kada brojač odbroji do svoje minimalne vrednosti, aktivira se dozvola za brojač desetica **TENS_CNT**.

Kako bi omogućili prelaz između brojača desetica i jedinica, potrebno je implementirati registar **PREV_CNT** koji čuva vrednost brojača jedinica.



Razmislite koji uslov mora biti zadovoljen da bi došlo do aktivacije signala **sgreen_tens_en**. U prethodnom pasusu opisane su neophodne komponente! Obratite pažnju na uslove kod brojača jedinica i kada se brojač resetuje.

Brojač desetica umanjuje svoju vrednost, a kad dođe do svoje minimalne vrednosti, prelazi se u stanje **YELLOW** i brojač **YELLOW_CNT** koji broji 1.5 sekund, nakon čega se prelazi u stanje **GREEN** i aktivira se crveno svetlo da pešake. Ukoliko se desi greška prilikom rada, sistem prelazi u stanje **ERROR** i ostaje do kraja.

Situacija je analogna za crveno svetlo za pešake.



Na prva dva displeja prikazuje se tajmer za zeleno svetlo za pešake, a na druga dva displeja za crveno. Vrednosti brojača za crveno i zeleno svetlo se šalju na **dekoder za 7-segmentni displej**, a izlaz iz dekodera na prikaz na displeju. Kako bi omogućili istovremeni prikaz na sva četiri displeja potrebno je realizovati vremensko multipleksiranje (**pogledati vežbe 5**).

Simulacija sistema:

Proveriti ponašanje sistema u simulaciji tako da se vide **dva ciklusa izmene stanja automata**. U okviru test bench datoteke tekstualno opisati u komentaru svaki test slučaj pre zadavanja signala u stimulus procesu.

Implementacija na ploču MAX1000 (OPCIONO):

Implementacija rešenja na platformi nije obavezna i služi kao dodatna provera rešenja. Za implementaciju na platformu MAX1000, potrebno je povezati prolaze na sledeći način:

- Ulaze **icLK** i **iRST** povezati na istoimene tastere
- Ulaz **iRUN** povezati na centralni taster
- Izlazi **oRED**, **oYELLOW**, **oGREEN** se vezuju na istoimene led diode
- **Semafor za pešake prikazati na sva četiri displeja istovremeno (videti sliku)**
- Signal **oDIS** se vezuje na pinove **J13** i **K1**



Kada je aktivno zeleno svetlo za pešake, prikaz na displejima za crveno svetlo (tajmer) ne treba biti vidljivi i obrnuto.

NAPOMENE ZA IZRADU ZADATKA:

- Rešenje realizovati u **dobijenoj postavci zadatka** (ne praviti novi projekat).
- **Nije dozvoljeno** menjati entitet sistema, kao i nazive već definisanih signala!
- Moguće je po potrebi dodati smislene interne signale.
- Na početku svake VHDL datoteke upisati **SVOJE PODATKE**.
- **OPCIONO** podesite pinove za svaki ulaz i izlaz sistema na osnovu podataka iz datoteke **LPRS1_FPGA_pins.pdf**.

NAPOMENE ZA PREDAJU REŠENJA NA SOVU:

- Realizovano rešenje postaviti na **SOVU** u predviđenom roku za svoju grupu po sledećim pravilima:
 - Potrebno je predati projekat **sa podešenim pinovima**.
 - Pre postavljanja rešenja uraditi **Clean projekta** (Project -> Clean Project).
 - ZIPOVATI **ceo projekat** u datoteku sa nazivom **LPRS1_Zad3_XX_YYY_QQQQ.zip**, gde je
 - XX – modul (RA, PSI ili IN),
 - YYY – trocifren broj indeksa (npr. 003 ako je indeks 3/2023)
 - QQQQ – godina upisa (npr. 2023)
 - **Primer** za indeks RA 3/2023 -> **LPRS1_Zad3_RA_003_2023**

- **Predati zadaci sa pogrešnim nazivom neće biti pregledani!!!**

NAPOMENE ZA ODBRANU ZADATKA:

- Odbrana zadatka se realizuje **na redovnom terminu računarskih vežbi**, gde student nakon nekoliko teorijskih pitanja i dodatnog zadatka može ostvariti **0, 1, 2 ili 5 poena**. Student koji je odgovorio tačno na sva pitanja i ispravno uradio dodatni zadatak u predviđenom vremenu može dobiti maksimum poena.
- Koraci odbrane domaćeg zadatka:
 - Student treba da dođe u **svoj termin računarskih vežbi** po rasporedu.
 - **Preuzeti** sa SOVE **svoje rešenje** – ceo projekat.
 - Pokrenuti **simulaciju** i **OPCIONO** implementirati rešenje **na platformu MAX1000**.
 - Nakon prethodno realizovanih koraka, pozvati asistenta za ispitivanje i dodatni zadatak.
- Student dobija **NULA POENA** u sledećim situacijama:
 - Student je predao rešenje na SOVU, ali se **nije pojavio** na odbrani zadatka,
 - Student **NIJE PREDAO** rešenje u **ZADATOM ROKU**,
 - Student je uredno predao rešenje, došao na odbranu, ali **ne pokazuje dovoljno znanja**,
 - Alat za proveru plagijarizma je pokazao **veliki procenat sličnosti** sa drugim rešenjima.